

868. Binary Gap

Solved 

Easy

 Topics

 Companies

Given a positive integer `n`, find and return the **longest distance** between any two **adjacent** `1`'s in the binary representation of `n`. If there are no two adjacent `1`'s, return `0`.

Two `1`'s are **adjacent** if there are only `0`'s separating them (possibly no `0`'s). The **distance** between two `1`'s is the absolute difference between their bit positions. For example, the two `1`'s in `"1001"` have a distance of 3.

Example 1:

Input: `n = 22`

Output: `2`

Explanation: 22 in binary is `"10110"`.

The first adjacent pair of `1`'s is `"10110"` with a distance of 2.

The second adjacent pair of `1`'s is `"10110"` with a distance of 1.

The answer is the largest of these two distances, which is 2. Note that `"10110"` is not a valid pair since there is a `1` separating the two `1`'s underlined.

Python:

class Solution:

def binaryGap(self, n: int) -> int:

if (n & (n - 1)) == 0: return 0

n //= n & -n

max_gap = 0

gap = 0

while n:

if n & 1:

max_gap = max(max_gap, gap)

gap = 0

else:

```
        gap += 1
        n >>= 1
```

```
    return max_gap + 1
```

JavaScript:

```
const binaryGap = n => {
    if (!(n & (n - 1))) return 0;
    n /= n & -n;
```

```
    let max = 0, gap = 0;
```

```
    while (n) {
        if (n & 1) {
            max = Math.max(max, gap);
            gap = 0;
        } else
            gap++;
        n >>= 1;
    }
```

```
    return max + 1;
```

```
};
```

Java:

```
class Solution {
    public int binaryGap(int n) {
        if ((n & (n - 1)) == 0) return 0;
        n >>= Integer.numberOfTrailingZeros(n);
        int maxGap = 0, gap = 0;

        while (n > 0) {
            if ((n & 1) == 1) {
                maxGap = Math.max(maxGap, gap);
                gap = 0;
            } else
                gap++;
            n >>= 1;
        }

        return maxGap + 1;
    }
}
```